

Project Report

Line Following Robot

EE.23. C.19 & 20

E/23/246

E/23/248

E/23/251

E/23/252

E/23/259

E/23/262



Introduction	2
The Logic of Digital Line Following Robot	3
Developing Logic to 4 Sensor Robot	5
Simulating in Proteus Software	7
Operating Principle	11
Key Components and Their Roles	11
How It Works	12
Frequency Equation	12
Building the Robot	13

—



Introduction

This project explores the fundamental principles of robotics by designing and implementing an autonomous line-following vehicle using purely hardware-based logic circuits. The objective of making this line following robot is to determine the control logic for our line following robot using only hardware components without microcontrollers.

This report documents our systematic development process—beginning with truth table analysis for 2, 3, and 4 sensor configurations, progressing through logic optimization, and providing valuable insights into the fundamentals of autonomous system design.

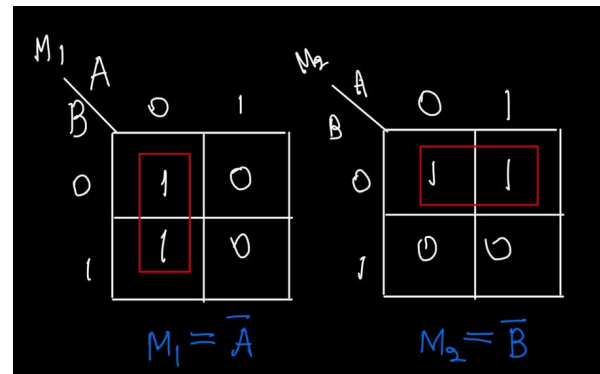
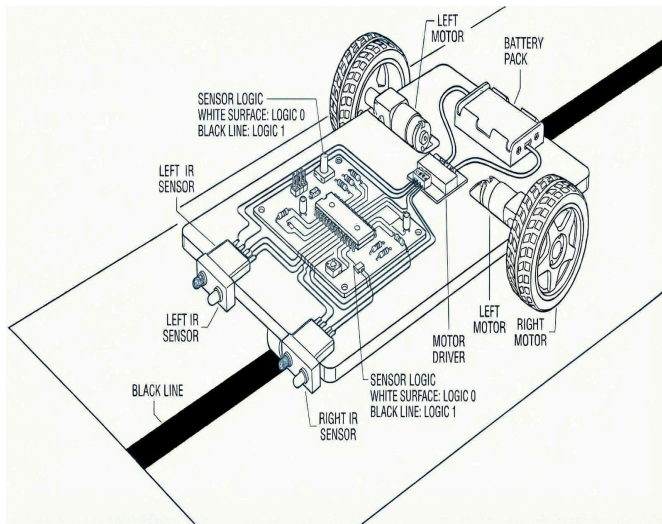
Through this hands-on exploration, we gain practical experience in sensor integration, analog-to-digital conversion, and control system implementation—skills directly transferable to more advanced robotic and automation applications.

The Logic of Digital Line Following Robot

Basic Line Following Robot (With 2 IR Sensors)

SENSOR ALIGNMENT : Both sensors are outside the black line.

LOGIC : Black = 1 , White = 0

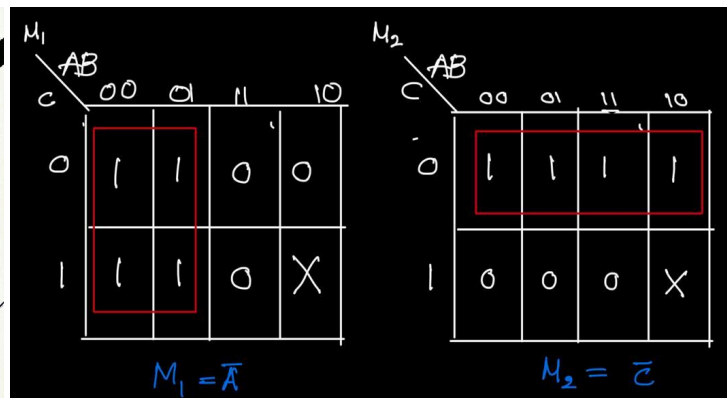
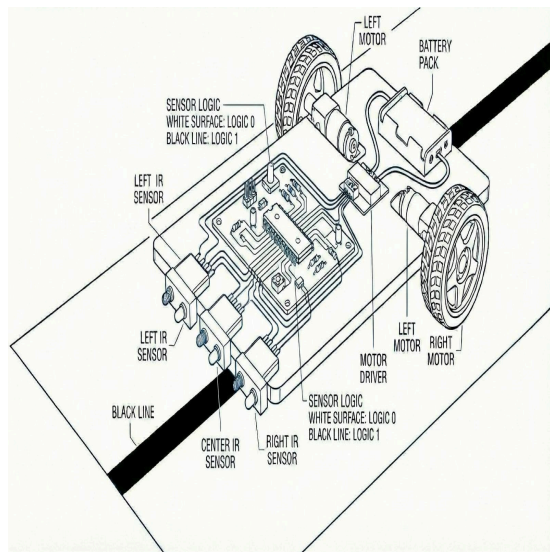


Sensor 1 (A)	Sensor 2 (B)	Motor 1	Motor 2
0	0	1	1
1	0	0	1
0	1	1	0
1	1	0	0

Line Following Robot with 3 IR Sensors

SENSOR ALIGNMENT : Only the middle sensor is above the black line, Other two are respectively on Left & Right.

LOGIC : Black = 1 , White = 0



Sensor 1 (A)	Sensor 2 (B)	Sensor 3 (C)	M1	M2
0	0	0	1	1
1	0	0	0	1
0	1	0	1	1
1	1	0	0	1
0	0	1	1	0
1	0	1	X	X
0	1	1	1	0
1	1	1	0	0

Developing Logic to 4 Sensor Robot

Line Following Robot with 4 IR Sensors

This line-following robot utilizes:

- 4 QTR sensors to detect the position of the line.
- Logic gates (AND, OR, NOT) to process the sensor signals.
- Two DC motors controlled via an H-Bridge Motor Driver (L293D).
- 555 Timer ICs to generate PWM signals for motor speed control.

The logic gates are implemented using 74HC series ICs, and the robot adjusts motor speed dynamically to align itself with the line.

The robot consists of two main sections:

1. Sensor Processing Unit:
 - 4 QTR sensors detect line position and output binary signals (0 or 1).
 - These sensor signals are fed into NOT gates (inverters), AND gates, and OR gates to implement the decision-making logic.
2. Motor Control Unit:
 - Two 555 Timer ICs generate PWM signals to control the motor speeds:
 - 33% PWM for slow speed.
 - 66% PWM for medium speed.
 - 100% PWM for full speed.
 - The H-Bridge Motor Driver (L293D) controls the direction and speed of the motors based on the logic output.

S 1	S 2	S 3	S 4	M1	M2	WHAT HAPPENS
-----	-----	-----	-----	----	----	--------------

0	0	0	0	0%	0%	No input - robot stops
0	1	1	0	100%	100%	Robot moves forward straight
0	0	1	0	100%	66%	Robot adjusts slightly to the right
0	1	0	0	66%	100%	Robot adjusts slightly to the left
0	0	1	1	100%	33%	Robot turns slowly to the right
1	1	0	0	33%	100%	Robot turns slowly to the left
1	1	1	0	0%	100%	Robot turns sharply left
0	1	1	1	100%	0%	Robot turns sharply right
1	0	0	0	0%	100%	Robot turns sharply left
0	0	0	1	100%	0%	Robot turns sharply right
1	1	1	1	0%	0%	Robot stops
1	0	0	1	100%	100%	Robot moves forward straight
1	0	1	0	100%	100%	Robot moves forward straight
0	1	0	1	100%	100%	Robot moves forward straight
1	1	0	1	100%	100%	Robot moves forward straight
1	0	1	1	100%	100%	Robot moves forward straight

S1	S2	S3	S4	Left Motor Forward	Left Motor Backward	Right Motor Forward	Right Motor Backward
0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	1
0	0	1	0	X	X	X	X
0	0	1	1	1	0	0	0
0	1	0	0	X	X	X	X
0	1	0	1	X	X	X	X
0	1	1	0	1	0	1	0
0	1	1	1	X	X	X	X
1	0	0	0	0	1	1	0
1	0	0	1	X	X	X	X
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	0	0	1	0
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

Left Motor Forward

	00	10	11	11
00	0 0	X 1	0 3	0 2
01	1 4	X 5	X 7	X 6
11	1 12	X 13	X 15	X 14
10	X 8	1 9	X 11	X 10

Left Motor Backward

	00	10	11	11
00	0 0	X 1	0 3	1 2
01	0 4	X 5	X 7	X 6
11	0 12	X 13	X 15	X 14
10	X 8	0 9	X 11	X 10

Right Motor Forward

	00	10	11	11
00	0 0	X 1	1 3	1 2
01	0 4	X 5	X 7	X 6
11	0 12	X 13	X 15	X 14
10	X 8	1 9	X 11	X 10

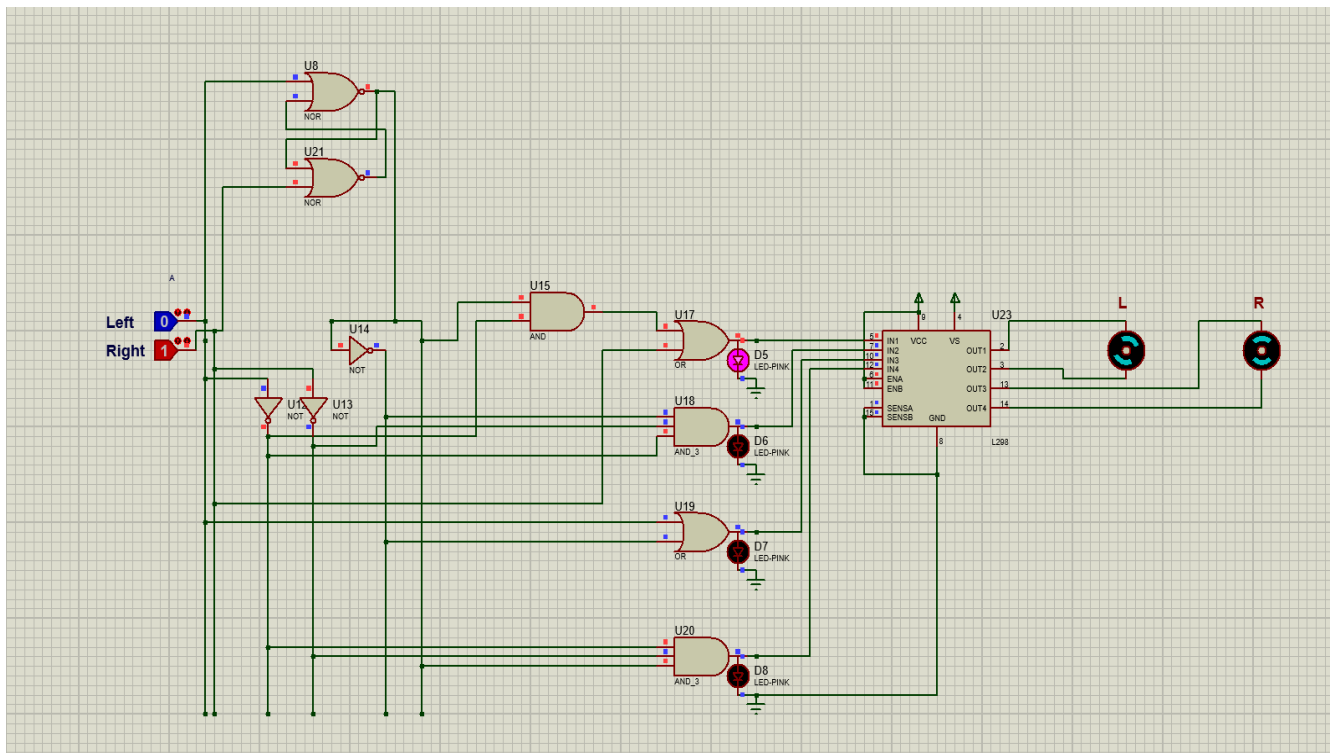
Right Motor Backward

	00	10	11	11
00	0 0	X 1	0 3	0 2
01	1 4	X 5	X 7	X 6
11	0 12	X 13	X 15	X 14
10	X 8	0 9	X 11	X 10

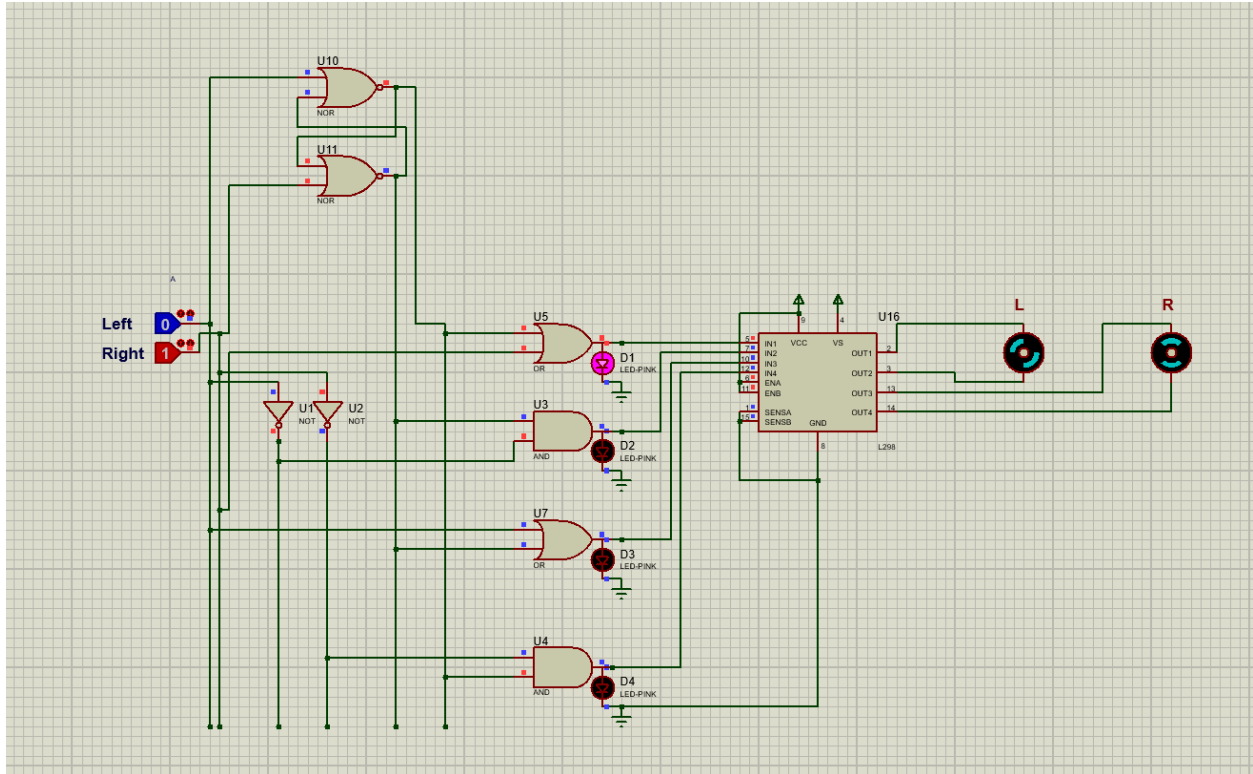
Simulating in Proteus Software

2 IR Sensors

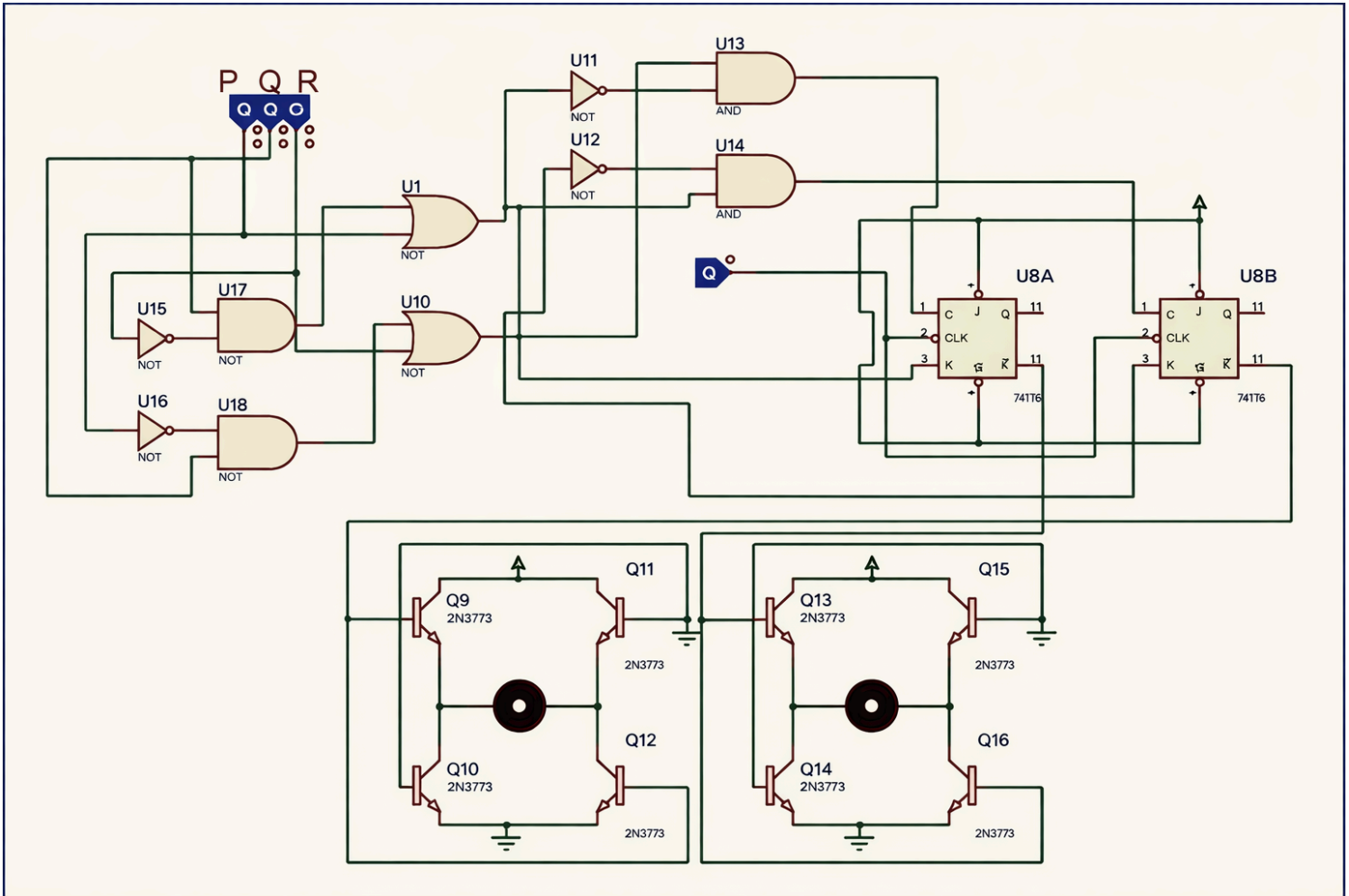
General Version



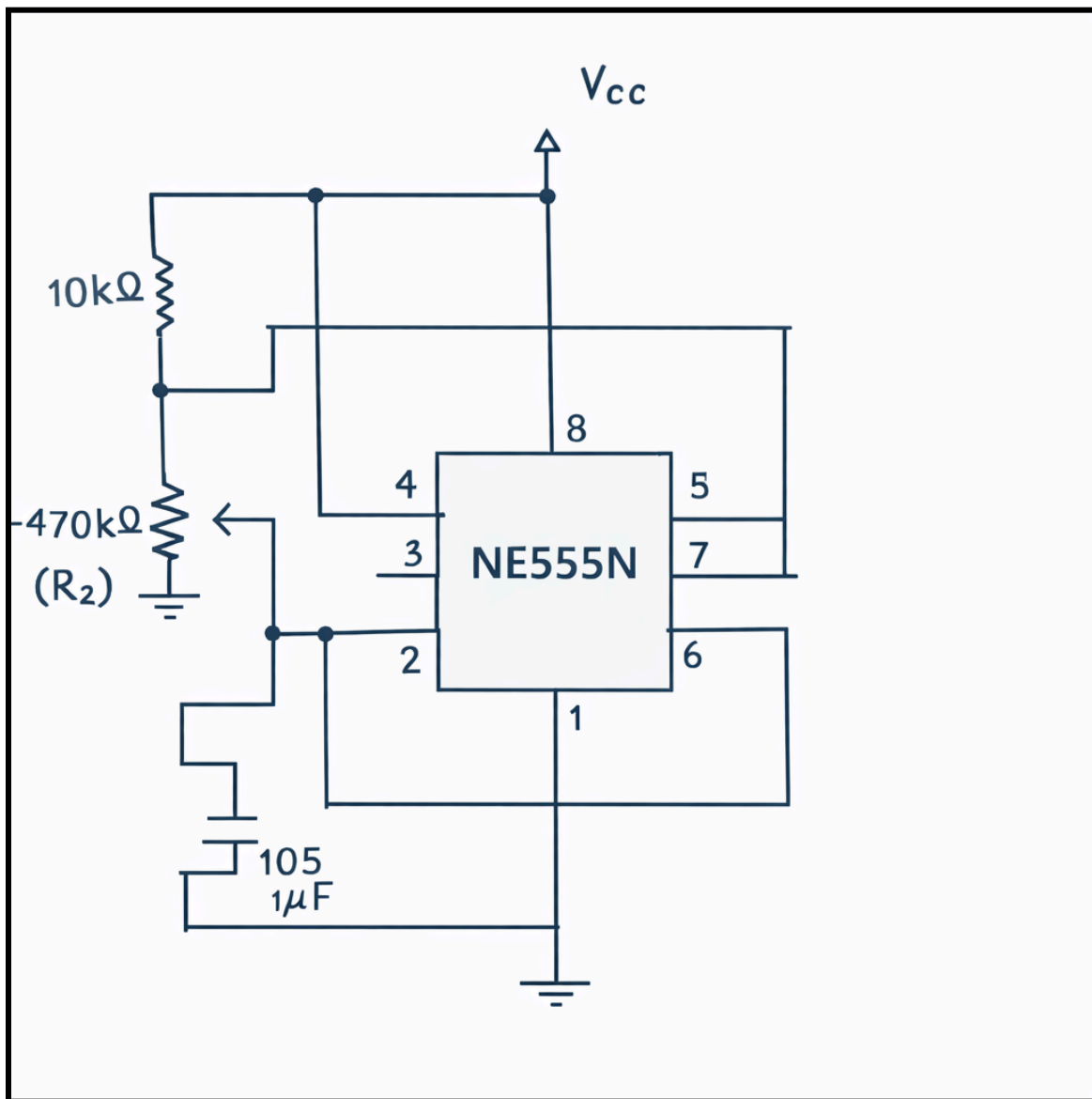
Simplified Version



3 IR Sensors



Clock Circuit



This is an **astable multivibrator clock circuit** built using the **NE555N timer IC**.

How we learned this.....

[Green Blinkenlight: Creating a Simple Clock Circuit with a 555 Timer](#)

Operating Principle

The 555 is configured in **astable mode**, meaning it continuously switches between HIGH and LOW without any external triggering, thereby generating a periodic square wave (clock signal).

Key Components and Their Roles

- **10 kΩ resistor (R1)**
Connected between VCC and pin 7 (Discharge). It limits charging current and helps determine frequency.
- **Variable resistor (0–470 kΩ, R2)**
Connected between pin 7 and the capacitor node (pins 2 & 6). It allows adjustment of the clock frequency.
- **Capacitor (105 = 1 μF)**
Connected between pins 2/6 and ground. It charges and discharges repeatedly to create oscillation.
- **NE555N connections**
 - *Pin 8 → VCC*
 - *Pin 1 → Ground*
 - *Pins 2 & 6 tied together (trigger + threshold)*
 - *Pin 7 → discharge path*
 - *Pin 3 → output (clock signal)*



How It Works

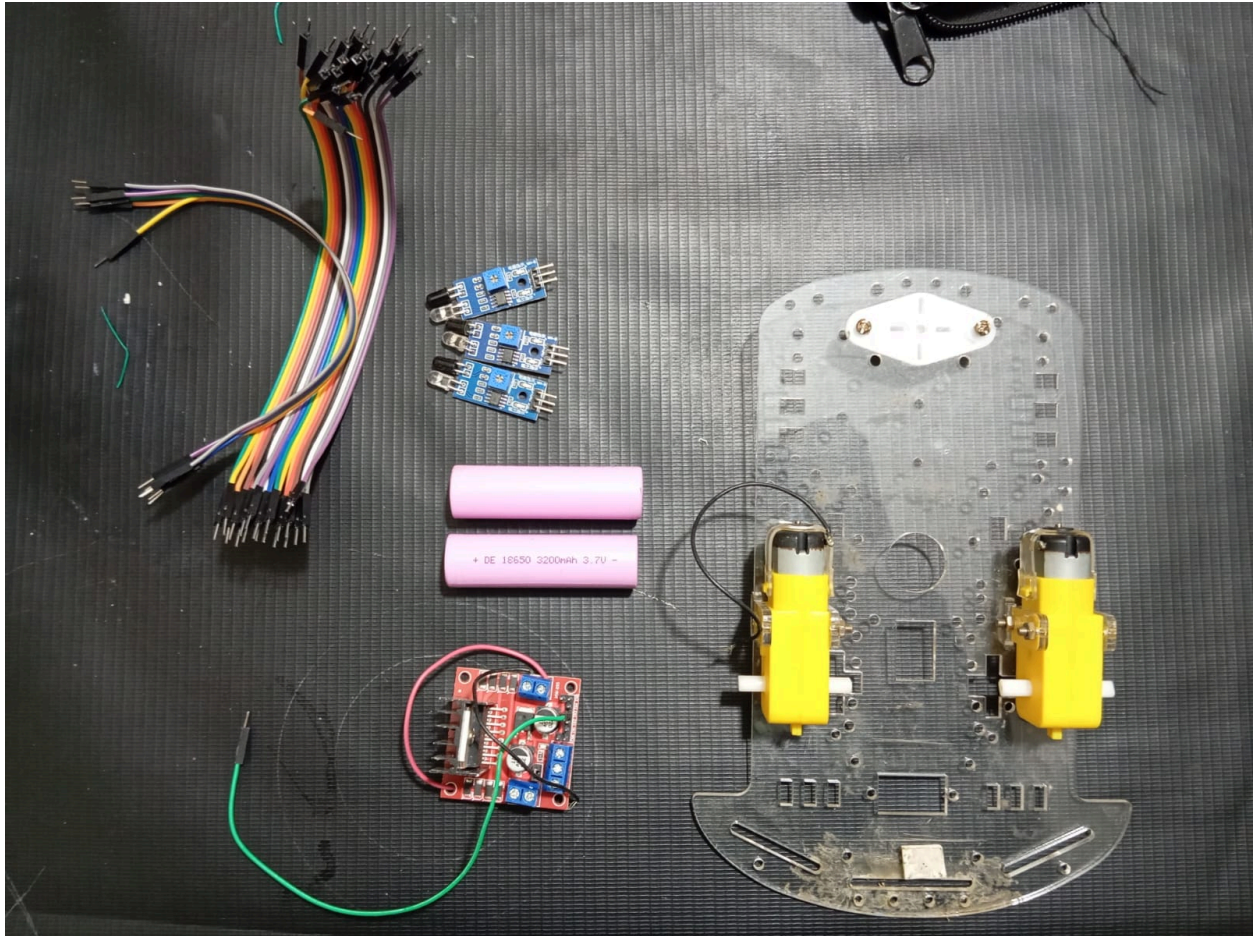
1. The capacitor charges through **R1 + R2**.
 2. When capacitor voltage reaches **2/3 VCC**, the internal comparator resets the flip-flop and turns ON the discharge transistor.
 3. The capacitor discharges through **R2**.
 4. When voltage drops to **1/3 VCC**, the output switches HIGH again.
 5. This cycle repeats continuously, producing a square wave at pin 3.
-

Frequency Equation

$$f = \frac{1.44}{(R_1 + 2R_2)C}$$

So the clock frequency is adjustable by changing **R2**.

Building the Robot



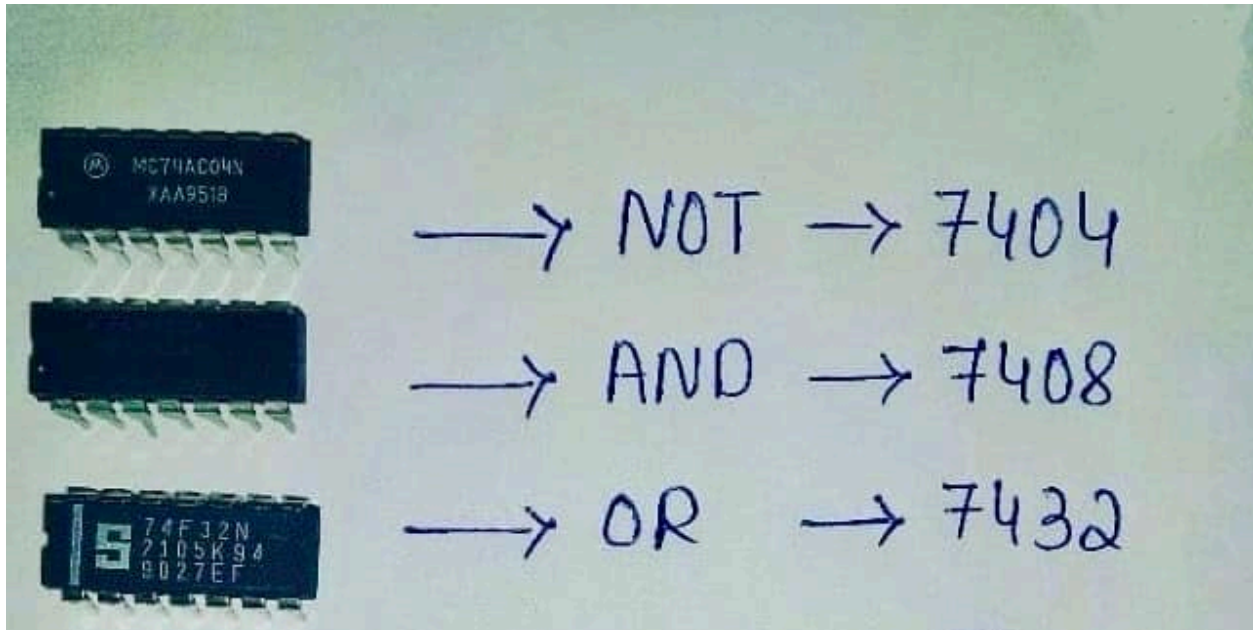
To be purchased.....



74LS73 Flip Flop (2PCS)



555 Timer IC (Adjustable)



7404 NOT GATES

7408 AND GATES

7432 OR GATES